

LABORATORIO 1

Ejercicio 3: Vendedores ordenados

Código Java y sección del servidor

Tema	Arreglos ordenados por nombre de vendedor
Operaciones	Alta, modificación y búsqueda
Archivos	N_Vendedor.java, Vendedor.java y Servidor.java

Enunciado

Una compañía quiere almacenar en arreglos ordenados la información de cada vendedor: nombre y total de ventas. Los arreglos permanecen ordenados alfabéticamente por el nombre del vendedor.

El programa permite dar de alta a un vendedor, modificar su total de ventas y listar el registro de un vendedor determinado.

Estructura de archivos

- N_Vendedor.java: clase que almacena los datos de un vendedor.
- Vendedor.java: clase principal con el arreglo ordenado y sus operaciones.
- Servidor.java: servidor HTTP que conecta la página HTML con la lógica Java.

1. Clase N_Vendedor.java

Crea esta clase en la misma carpeta o paquete del proyecto Java.

```
public class N_Vendedor {
    private String nombre;
    private float totalVentas;

    public N_Vendedor(String nombre, float totalVentas) {
        this.nombre = nombre;
        this.totalVentas = totalVentas;
    }

    public String getNombre() { return nombre; }
    public float getTotalVentas() { return totalVentas; }
    public void setTotalVentas(float totalVentas) { this.totalVentas = totalVentas; }

    @Override
    public String toString() {
        return nombre + " | " + totalVentas;
    }
}
```

2. Clase Vendedor.java

El arreglo se mantiene ordenado alfabéticamente por nombre. La búsqueda es binaria porque el arreglo ya está ordenado.

```
public class Vendedor {
    private N_Vendedor[] vendedores = new N_Vendedor[100];
    private int tam = 0;

    private int buscar(String nombre) {
        int inicio = 0, fin = tam - 1;
        while (inicio <= fin) {
            int medio = (inicio + fin) / 2;
            int comparacion = vendedores[medio].getNombre().compareToIgnoreCase(nombre);
            if (comparacion == 0) return medio;
            if (comparacion < 0) inicio = medio + 1;
            else fin = medio - 1;
        }
        return -1;
    }

    public String alta(String nombre, float ventas) {
        if (tam == vendedores.length) return "No hay espacio para más vendedores";
        if (buscar(nombre) != -1) return "El vendedor ya existe";
        int pos = 0;
        while (pos < tam && vendedores[pos].getNombre().compareToIgnoreCase(nombre) < 0) pos++;
        for (int i = tam; i > pos; i--) vendedores[i] = vendedores[i - 1];
        vendedores[pos] = new N_Vendedor(nombre, ventas);
        tam++;
        return "Vendedor agregado correctamente";
    }

    public String modificarVentas(String nombre, float ventas) {
        int pos = buscar(nombre);
        if (pos == -1) return "El vendedor no existe";
        vendedores[pos].setTotalVentas(ventas);
        return "Ventas modificadas correctamente";
    }

    public String listarUno(String nombre) {
        int pos = buscar(nombre);
        return pos == -1 ? "El vendedor no existe" : vendedores[pos].toString();
    }
}
```

3. Sección para Servidor.java

Agrega esta variable, rutas y manejadores al servidor actual.

```
private static Vendedor vendedores = new Vendedor();
servidor.createContext("/ordenado3/alta", new AltaVendedorHandler());
servidor.createContext("/ordenado3/modificar", new ModificarVendedorHandler());
servidor.createContext("/ordenado3/listarUno", new ListarUnoVendedorHandler());

static class AltaVendedorHandler implements HttpHandler {
    public void handle(HttpExchange e) throws IOException {
        Map<String,String> d = leerDatos(e);
        responder(e, vendedores.alta(d.get("nombre"), Float.parseFloat(d.get("ventas"))));
    }
}

static class ModificarVendedorHandler implements HttpHandler {
    public void handle(HttpExchange e) throws IOException {
        Map<String,String> d = leerDatos(e);
```

```
        responder(e, vendedores.modificarVentas(d.get("nombre"), Float.parseFloat(d.get("ventas"))));
    }
}
static class ListarUnoVendedorHandler implements Handler {
    public void handle(HttpExchange e) throws IOException {
        responder(e, vendedores.listarUno(LeerDatos(e).get("nombre")));
    }
}
```

Rutas que utilizará la página HTML

La sección JavaScript del ejercicio debe enviar los datos con método POST a estas rutas:

Operación	Ruta del servidor
Dar de alta	/ordenado3/alta
Modificar ventas	/ordenado3/modificar
Listar un vendedor	/ordenado3/listarUno

Importante para integrarlo

- Los campos enviados desde HTML deben ser: nombre y ventas.
- El arreglo no usa base de datos: los datos existen mientras el servidor Java está ejecutándose.
- La inserción desplaza los elementos necesarios para conservar el orden alfabético por nombre.
- La búsqueda del vendedor se realiza con búsqueda binaria porque el arreglo permanece ordenado.



Ejercicio 3

Vendedores ordenados por nombre

[Abrir ejercicio](#)[Documento](#)

Arreglos Ordenados: Ejercicio 3

Menú de Opciones

[🌙 Modo Oscuro](#)

Configurar arreglo

Tamaño del arreglo:

Ejemplo: 5

[Iniciar ejercicio](#)[Salir](#)

Arreglos Ordenados: Ejercicio 3

Menú de Opciones

[🌙 Modo Oscuro](#)[1. Dar de alta](#)[2. Modificar total de ventas](#)[3. Listar un vendedor](#)[4. Salir](#)

Arreglos Ordenados: Ejercicio 3

Menú de Opciones

[🌙 Modo Oscuro](#)[1. Dar de alta](#)[2. Modificar total de ventas](#)[3. Listar un vendedor](#)[4. Salir](#)

Dar de alta

Nombre del vendedor:

Total de ventas:

[Guardar](#)

Arreglos Ordenados: Ejercicio 3

Menú de Opciones

[🌙 Modo Oscuro](#)[1. Dar de alta](#)[2. Modificar total de ventas](#)[3. Listar un vendedor](#)[4. Salir](#)

Modificar total de ventas

Nombre del vendedor:

Nuevo total de ventas:

[Actualizar](#)



Arreglos Ordenados: Ejercicio 3

Menú de Opciones

🌙 Modo Oscuro

1. Dar de alta

2. Modificar total de ventas

3. Listar un vendedor

4. Salir

Listar un vendedor

Nombre del vendedor:

Buscar